

DSA STUDY BLUEPRINT SERIES

# 100. Largest BST in Binary Tree

Finding Largest BST Subtree in Binary Trees

Section 09 • C++ Core Data Structures & Algorithms

# Core Concepts & Visual Blueprint

Theoretical models and memory architectures

---

## Key Principles

- **Problem:** Return size of largest valid BST subtree.
- Pass node min/max bounds and valid state indicators bottom-up.

## Memory & Execution Blueprint

Child subtrees valid checks: `left.isBST && right.isBST && left.max < root < right.min`

# C++ Implementation Reference

Standard code layout and syntax

---

```
struct BSTInfo {  
    bool isBST; int sz, minVal, maxVal;  
};  
// Return info containing child bounds. Valid if left.max < root < right.min.
```

# Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

## Performance Table

| Aspect / Case    | Complexity |
|------------------|------------|
| Time Complexity  | $O(N)$     |
| Space Complexity | $O(H)$     |

## Key Practices & Gotchas

- **Important:** Pass bounds bottom-up to achieve  $O(N)$  runtime, avoiding repeated validator searches.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

# Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

## Execution Walkthrough

Let's trace BST constraint validation checks verifying node ranges `[Min, Max]`:

| Active Node | Valid range limit    | Comparison check                | Recursive branch step                         |
|-------------|----------------------|---------------------------------|-----------------------------------------------|
| 15 (Root)   | $[-\infty, +\infty]$ | True $(-\infty < 15 < +\infty)$ | Recurse Left child with range $[-\infty, 15]$ |
| 10 (Left)   | $[-\infty, 15]$      | True $(-\infty < 10 < 15)$      | Recurse Right child with range $[10, 15]$     |

# Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

---

## Key Variations & Follow-up Questions

- **Inorder property:** BST Inorder traversals yield strictly increasing sorted lists.
- **Tree Balancing:** Balanced tree heights (AVL/Red-black) guarantee logarithmic search times.
- **Related LeetCode Problems:** #98 Validate Binary Search Tree, #700 Search in a Binary Search Tree, #108 Convert Sorted Array to Binary Search Tree.